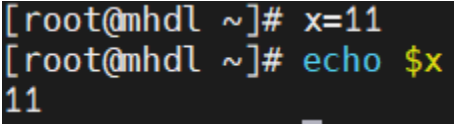
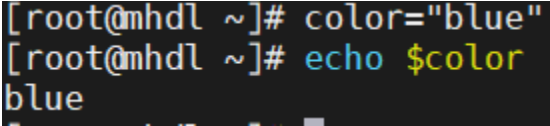
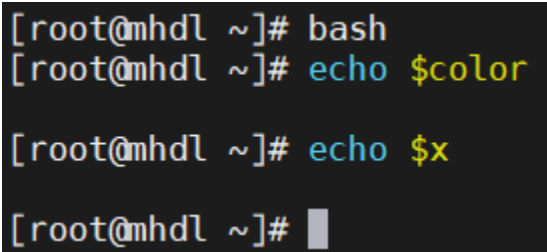


Task 1

- 1) We need to pay attention to exit code of last command. So, we just need to run this command:

``echo $?`` - If its output is 0, that refers to the last command was successful.

However, it is another number than 0, that means there was an error. We can search for this number of exit code and find out what is the problem.

- 2) 
- 3) 
- 4) 

As you can see, in another session of terminal the previous variables cannot be found.

- 5) We just need to open `/etc/profile` or `/etc/bashrc` file and add this line to end of the file:

`export mydata=/opt/mydata`. Then save and quit the file.

- 6) We need to run `source /etc/profile` command.

- 7) `~/bashrc`, `~/bash_profile`

- 8) `/etc/profile`, `/etc/bashrc`

```
[root@mhd1 ~]# echo "export myVar=123" >> /etc/bashrc
[root@mhd1 ~]# echo "export myVar=890" >> ~/.bashrc
[root@mhd1 ~]# tail -n1 /etc/bashrc ; tail -n1 ~/.bashrc
export myVar=123
export myVar=890
[root@mhd1 ~]# bash
[root@mhd1 ~]# echo $myVar
890
```

9)

As you can see, the user specific variable wins. Because initially the global variables are loaded, then user specific variables. Thus, user specific ones override the global ones. However, we need to put the variables to end of the file, especially in bashrc, it is better to prevent system confusions.

10) We can use ``env`` or ``printenv`` command for it.

11)

```
[root@mhd1 ~]# mkdir -p /usr/java/java_11/bin #Creating_Folders
[root@mhd1 ~]# touch /usr/java/java_11/bin/java #Creating_File
[root@mhd1 ~]# chmod o+x /usr/java/java_11/bin/java #Giving_Execute_Permission
[root@mhd1 ~]# useradd apple && passwd apple && su - apple
Changing password for user apple.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[apple@mhd1 ~]$ vi ~/.bash_profile
```

First, we create a ``/usr/java/java_11/bin`` folder and ``java`` file inside it. To be able to create anything in ``/usr/``, we need to have permissions, so I just did it as a root.

Then I created a user and gave him password and switched it to him.

Then opened its ``~/.bash_profile`` file and add these lines:

```
export JAVA_HOME=/usr/java/java_11
export PATH=$JAVA_HOME/bin:$PATH
```

Then I need to re-login or just run ``source ~/.bash_profile`` to allow system to read the changes in ``~/.bash_profile`` file.

Then we can just run and check:

```
[apple@mhd1 ~]$ which java
/usr/java/java_11/bin/java
```

Task 2

- 1) An inode is a data structure in Linux file systems that stores info about files, but not the file name and not the actual data itself. It stores:
 - i. File type (whether it is a regular file or directory or more)
 - ii. Its permissions
 - iii. The count of hard links
 - iv. User and primary
 - v. Timestamps (created or modified time)
 - vi. Pointers to the disk blocks (clusters) where the real data is stored

```
[root@mhd1 WorkingDirectory]# dd if=/dev/zero of=testfile bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.261474 s, 401 MB/s
[root@mhd1 WorkingDirectory]# ls
testfile
[root@mhd1 WorkingDirectory]# ln testfile hard_link_1
[root@mhd1 WorkingDirectory]# ln testfile hard_link_2
[root@mhd1 WorkingDirectory]# ln testfile hard_link_3
[root@mhd1 WorkingDirectory]# ln testfile hard_link_4
2) [root@mhd1 WorkingDirectory]# ln testfile hard_link_5
```

```
[root@mhd1 WorkingDirectory]# ls -lih
total 600M
26015084 -rw-r--r--. 6 root root 100M Feb 26 03:07 hard_link_1
26015084 -rw-r--r--. 6 root root 100M Feb 26 03:07 hard_link_2
26015084 -rw-r--r--. 6 root root 100M Feb 26 03:07 hard_link_3
26015084 -rw-r--r--. 6 root root 100M Feb 26 03:07 hard_link_4
26015084 -rw-r--r--. 6 root root 100M Feb 26 03:07 hard_link_5
26015084 -rw-r--r--. 6 root root 100M Feb 26 03:07 testfile
```

I add `-h` option to make it more human readable.

7)

a) Overall, only 100 MB size, because creating hard links does not allocate new space from the dist, they just are connected to inode of main file. That is why space stays the same.

b) Nothing happens, hard links stay the same and keep the data. The data is completely lost only in the main file, and all its hard links are deleted.

c) File type; permissions; hard link count; user and group; timestamps (created and modified time); pointers to disc blocks where the real data is stored.

d) A hard link points directly to the inode of a file. The inode is stored inside the filesystem of that disk (or partition). Inodes are unique only inside the same filesystem, so you cannot link across filesystems/disks.

e)

```
[root@mhdl opt]# ln /boot/config-5.14.0-611.5.1.el9_7.x86_64 hard_link  
ln: failed to create hard link 'hard_link' => '/boot/config-5.14.0-611.5.1.el9_7.x86_64': Invalid cross-device link
```

/boot is on one filesystem (one “disk area”)

/opt is on another filesystem (often a separate partition)

Hard links point directly to the inode of a file, which exists only inside the same filesystem. So we cannot create hard link of file in different disks.

f)

```
[root@mhd1 WorkingDirectory]# ll
total 4
-rw-r--r--. 1 root root 20 Feb 26 03:28 mainfile.txt
[root@mhd1 WorkingDirectory]# ln -s mainfile.txt soft_link_1
[root@mhd1 WorkingDirectory]# ln -s mainfile.txt soft_link_2
[root@mhd1 WorkingDirectory]# ll
total 4
-rw-r--r--. 1 root root 20 Feb 26 03:28 mainfile.txt
lrwxrwxrwx. 1 root root 12 Feb 26 03:28 soft_link_1 -> mainfile.txt
lrwxrwxrwx. 1 root root 12 Feb 26 03:28 soft_link_2 -> mainfile.txt
[root@mhd1 WorkingDirectory]# rm -f mainfile.txt
[root@mhd1 WorkingDirectory]# ll
total 0
lrwxrwxrwx. 1 root root 12 Feb 26 03:28 soft_link_1 -> mainfile.txt
lrwxrwxrwx. 1 root root 12 Feb 26 03:28 soft_link_2 -> mainfile.txt
```

When we remove original file, the soft links become meaningless and data becomes removed. When we try to cat softlinks:

```
[root@mhd1 WorkingDirectory]# cat soft_link_1
cat: soft_link_1: No such file or directory
[root@mhd1 WorkingDirectory]# cat soft_link_2
cat: soft_link_2: No such file or directory
```